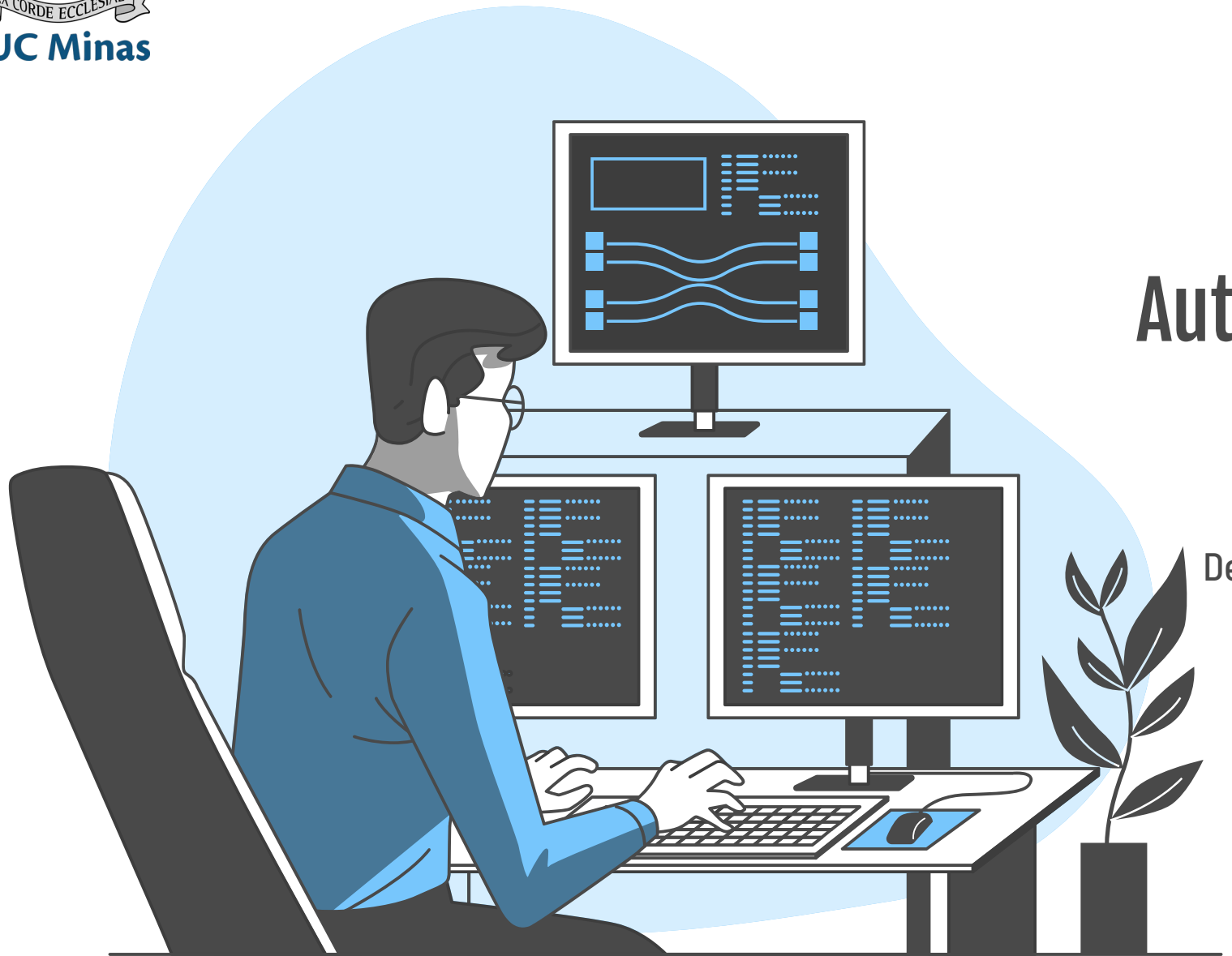
PUC Minas



Aula 08 · FullStack

Autenticação full stack com JWT

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Implementar o fluxo completo de login entre React e Spring Boot
2. Guardar o token e conhecer os riscos de `localStorage`
3. Enviar `Authorization: Bearer` em todas as requisições pelo `httpClient`
4. Criar um **AuthContext** com `login`, `logout` e `usuario`
5. Tratar `401` e `403` no front

Antes de começar

- Back-end, aulas 09 e 10
- Front-end, aula 08
- Aulas 03 e 04 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

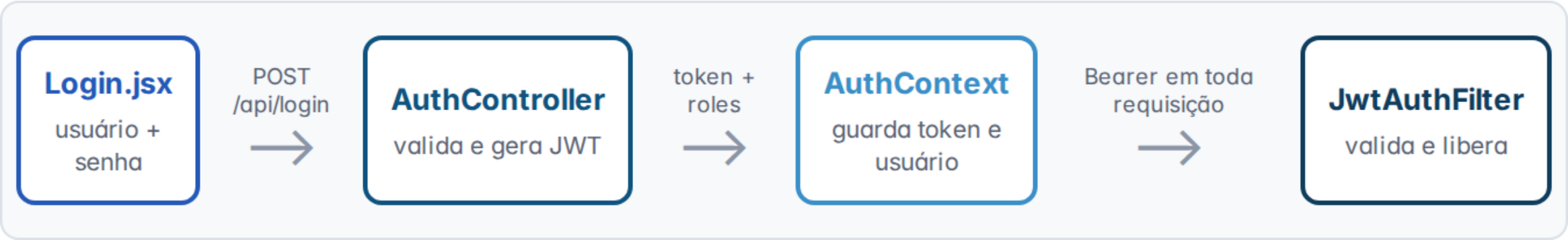
01 Back-end: o que precisa estar pronto

02 Front-end

03 Conferindo no navegador

04 Aprofundando: sessão no front

Introdução



O ciclo completo do login com JWT.

01

Back-end: o que precisa estar pronto

Back-end: o que precisa estar pronto

Do Back-end, aula 10: `POST /api/login` devolvendo `{ token, roles }`, o `JwtAuthFilter` registrado, sessão `STATELESS` e CSRF desligado. Para o navegador, falta um detalhe: o CORS precisa aceitar o cabeçalho **`Authorization`**, porque ele dispara o *preflight*.

Back: SecurityConfig.java

```
http
    .csrf(csrf -> csrf.disable())
    .cors(Customizer.withDefaults())                        // usa o CorsConfigurationSource
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()   // preflight
        .requestMatchers(HttpMethod.POST, "/api/login").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/produtos").permitAll()
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated())
    .exceptionHandling(e -> e.authenticationEntryPoint(
        (req, res, ex) -> res.sendError(401, "Token ausente ou inválido.")))
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

// no CorsConfigurationSource:
cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
```

02

Front-end

1. Onde guardar o token

Lugar	Sobrevive ao F5?	Risco
Estado do React (memória)	Não	O mais seguro; o usuário loga de novo ao recarregar
`sessionStorage`	Sim, até fechar a aba	Lido por qualquer script da página (XSS)
`localStorage`	Sim, até sair	Igual ao anterior; o mais usado em projetos didáticos
Cookie `HttpOnly` (setado pelo back)	Sim	JavaScript não lê; exige CSRF e configuração de cookies entre domínios

1. Onde guardar o token

Nos projetos da disciplina usamos `localStorage` pela simplicidade. Em produção, o cuidado principal é **evitar XSS**: nunca use `dangerouslySetInnerHTML` com dados do usuário.

02

Front-end

Aula 08 · FullStack

2. httpClient com o token

src/services/httpClient.js

```
import API_CONFIG from "../config/apiConfig";
import { lerToken, limparSessao } from "../utils/storage";

export async function requisicao(caminho, opcoes = {}) {
  const token = lerToken();
  const headers = {
    ...(opcoes.body ? { "Content-Type": "application/json" } : {}),
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  };

  let resposta;
  try {
    resposta = await fetch(`${API_CONFIG.BASE_URL}${caminho}`, { ...opcoes, headers });
  } catch {
    throw new Error("Não foi possível falar com o back-end.");
```

2. httpClient com o token (cont.)

src/services/httpClient.js

```
  }

  if (resposta.status === 401) {          // token expirado ou inválido
    limparSessao();
    window.dispatchEvent(new Event("sessao-expirada"));
    throw new Error("Sua sessão expirou. Entre novamente.");
  }
  if (resposta.status === 403) throw new Error("Você não tem permissão para esta ação.");
  if (resposta.status === 204) return null;

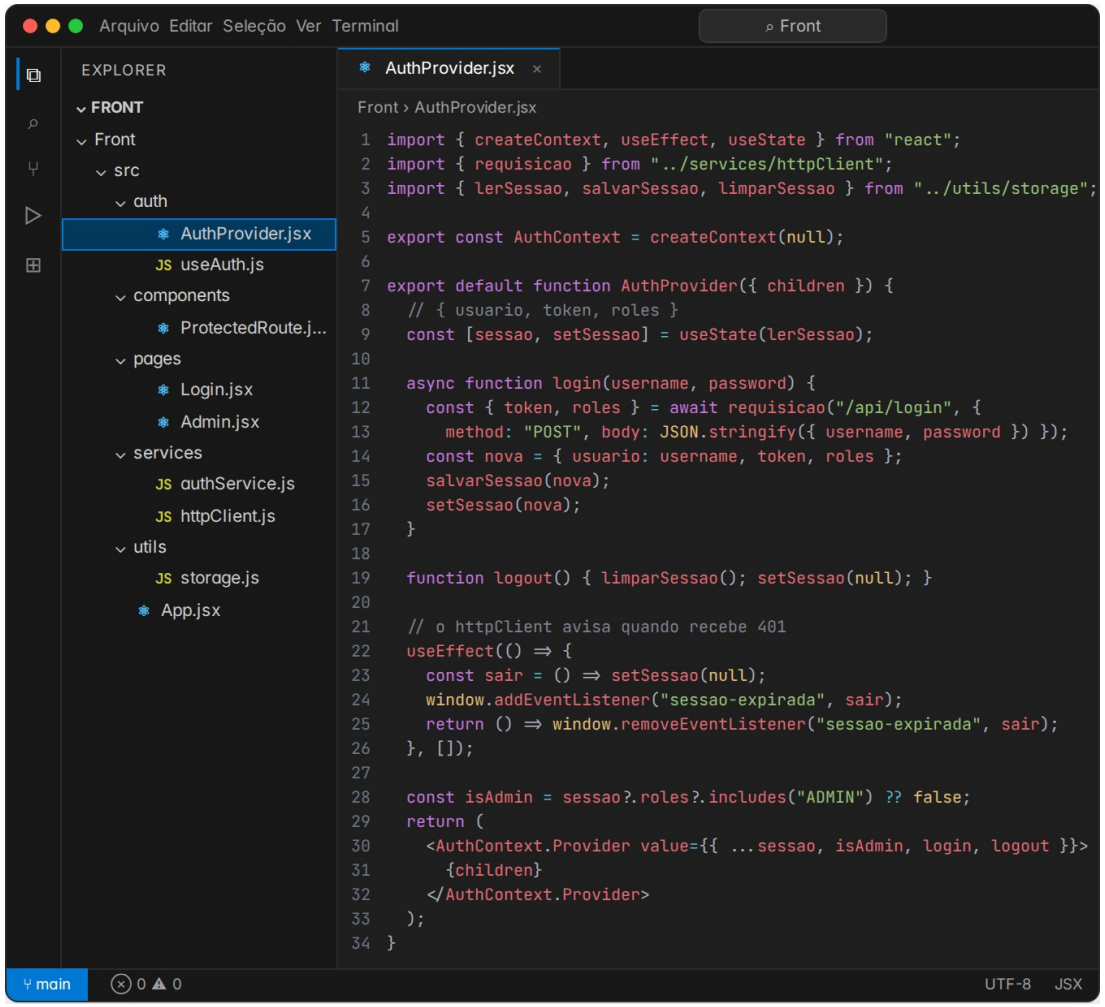
  const corpo = await resposta.json().catch(() => null);
  if (!resposta.ok) throw new Error(corpo?.mensagem || `Erro ${resposta.status}.`);
  return corpo;
}
```

2. httpClient com o token

src/utils/storage.js

```
const CHAVE = "auth";
export const salvarSessao = (s) => localStorage.setItem(CHAVE, JSON.stringify(s));
export const lerSessao = () => JSON.parse(localStorage.getItem(CHAVE) || "null");
export const lerToken = () => lerSessao()?.token ?? null;
export const limparSessao = () => localStorage.removeItem(CHAVE);
```

3. AuthContext



`AuthProvider`: um *Context* deixa o usuário logado disponível em qualquer componente.

3. AuthContext

src/auth/useAuth.js e main.jsx

```
import { useContext } from "react";
import { AuthContext } from "./AuthProvider";
export const useAuth = () => useContext(AuthContext);

// main.jsx
<AuthProvider>
  <App />
</AuthProvider>
```

4. Tela de login e rota protegida

pages/Login.jsx

```jsx
export default function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [dados, setDados] = useState(
    { username: "", password: "" });
  const [erro, setErro] = useState("");

  async function entrar(e) {
    e.preventDefault();
    try {
      await login(dados.username, dados.password);
      navigate("/admin");
    } catch (err) {
      setErro("Usuário ou senha inválidos.");
    }
  }
  // ... formulário controlado
}
```

components/ProtectedRoute.jsx

```jsx
export default function ProtectedRoute({
    children, somenteAdmin = false }) {
  const { token, isAdmin } = useAuth();

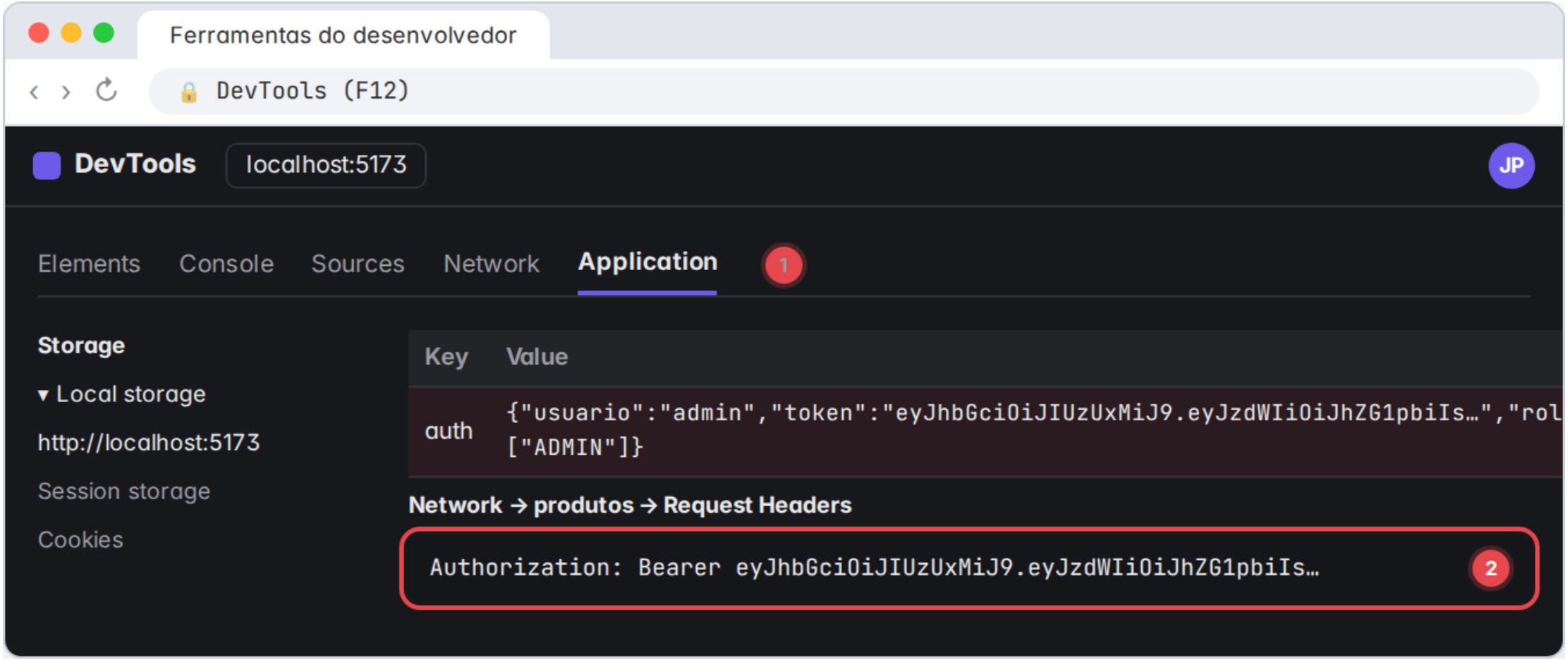
  if (!token)
    return <Navigate to="/login" replace />;
  if (somenteAdmin && !isAdmin)
    return <Navigate to="/" replace />;
  return children;
}

// App.jsx
<Route path="/admin" element={
  <ProtectedRoute somenteAdmin>
    <Admin />
  </ProtectedRoute>} />
```

03

Conferindo no navegador

Conferindo no navegador



Aba **Application**: a sessão no `localStorage`. Na aba **Network**, o cabeçalho `Authorization` enviado pelo `httpClient`.

Conferindo no navegador

Erro comum

Login funciona no Insomnia e falha no React com erro de CORS: o *preflight* do `POST /api/login` foi barrado (falta `.cors(...)` ou o `OPTIONS` liberado). **Logo após o login, toda chamada dá 401**: o token não está indo (confira o cabeçalho na aba Network) ou o back reiniciou com outra chave secreta (use `JWT_SECRET` fixo, Back-end aula 10). **403 em `/api/admin`**: o usuário não tem `ADMIN`.

04

Aprofundando: sessão no front

Aprofundando: sessão no front

Lendo o exp do JWT no front (só para UX: quem valida é o back)

```
function tokenExpirado(token) {
  try {
    const payload = JSON.parse(atob(token.split(".")[1].replace(/-/g, "+").replace(/_/g, "/")));
    return payload.exp * 1000 < Date.now();
  } catch {
    return true;
  }
}

// ao abrir o app: descarta sessão vencida
const inicial = lerSessao();
const [sessao, setSessao] = useState(inicial && !tokenExpirado(inicial.token) ? inicial : null);
```

Aprofundando: sessão no front

1. Login devolve o token; o `AuthProvider` guarda em memória e no `localStorage`.
2. Ao abrir o app, o provider lê a sessão salva e descarta se estiver expirada.
3. Toda requisição leva `Authorization: Bearer` pelo `httpClient`.
4. Recebeu 401? O `httpClient` limpa a sessão e dispara `sessao-expirada`; o app volta ao login.
5. Logout limpa a sessão e navega para `/login`.

O que o front decide x o que o back decide

Front (experiência)

- Esconder botões de quem não é ADMIN
- Redirecionar para o login
- Mostrar “sessão expirada”

vs

Back (segurança)

- Validar assinatura e expiração do token
- Negar com 401/403
- Nunca confiar em `roles` enviadas pelo cliente

Aprofundando: sessão no front

Componente que mostra conteúdo só para ADMIN

```
function SoAdmin({ children }) {
  const { isAdmin } = useAuth();
  return isAdmin ? children : null;
}

<SoAdmin>
  <button onClick={() => excluir(p)}>Excluir</button>
</SoAdmin>
```

Aprofundando: sessão no front

Status	Significado no login JWT	Reação do front
200 no `/api/login`	Credenciais corretas	Guarda o token e navega
401 no `/api/login`	Usuário ou senha errados	Mensagem genérica no formulário
401 em outra rota	Token ausente, inválido ou expirado	Limpa a sessão e vai ao login
403	Logado sem permissão	Mensagem “sem permissão”, mantém logado

Aprofundando: sessão no front

Atenção

Nunca envie a senha em `GET` ou na URL (`/api/login?senha=...`): URLs ficam no histórico do navegador e nos logs dos servidores. Login é sempre `POST` com JSON no corpo, sempre por HTTPS em produção.

05

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que precisa constar no CORS para o login JWT funcionar?

A Só GET

B Permitir o cabeçalho Authorization e o OPTIONS

C allowedOrigins("*")

D Nada

Resposta

O que precisa constar no CORS para o login JWT funcionar?

A Só GET

✓ Permitir o cabeçalho Authorization e o OPTIONS

C allowedOrigins("*")

D Nada

Por quê: O cabeçalho dispara o preflight.

Pergunta

Qual o principal risco de guardar o token no localStorage?

A Fica lento

B Scripts injetados (XSS) podem lê-lo

C Expira mais rápido

D Não funciona no Vercel

Resposta

Qual o principal risco de guardar o token no localStorage?

A Fica lento

✓ Scripts injetados (XSS) podem lê-lo

C Expira mais rápido

D Não funciona no Vercel

Por quê: Evite XSS; não use dangerouslySetInnerHTML com dados do usuário.

Pergunta

O que o httpClient faz ao receber 401?

A Ignora

B Limpa a sessão e avisa o app para voltar ao login

C Repete para sempre

D Mostra 404

Quiz · questão 3 de 5

Resposta

O que o httpClient faz ao receber 401?

A Ignora

✓ Limpa a sessão e avisa o app para voltar ao login

C Repete para sempre

D Mostra 404

Por quê: Evento sessao-expirada.

Pergunta

Para que serve o AuthContext?

A Guardar CSS

B Disponibilizar usuário, token, login e logout para todo o app

C Configurar o Vite

D Conectar ao banco

Quiz · questão 4 de 5

Resposta

Para que serve o AuthContext?

A Guardar CSS

✓ Disponibilizar usuário, token, login e logout para todo o app

C Configurar o Vite

D Conectar ao banco

Por quê: Lido com o hook useAuth().

Pergunta

Esconder o botão Excluir de quem não é ADMIN garante segurança?

A Sim

B Não: o back também precisa negar com 403

C Só com HTTPS

D Só no Render

Resposta

Esconder o botão Excluir de quem não é ADMIN garante segurança?

A Sim

✓ Não: o back também precisa negar com 403

C Só com HTTPS

D Só no Render

Por quê: O front decide a experiência; o back decide a segurança.

Fechamento

Resumo da aula

- ✓ Back: `/api/login` público, filtro JWT, `STATELESS`, CORS aceitando `Authorization` e `OPTIONS` liberado.
- ✓ Front: o token fica no `localStorage` (didático) ou em memória (mais seguro).
- ✓ O `httpClient` injeta `Authorization: Bearer` e trata `401` (sai) e `403` (sem permissão).
- ✓ Um `AuthContext` expõe `usuario`, `token`, `isAdmin`, `login` e `logout` para todo o app.
- ✓ `ProtectedRoute` controla a navegação; o back continua sendo quem protege os dados.

Exercícios

1. Adicione login ao cardápio: listar é público, cadastrar/editar/excluir exige login.

2. Mostre o botão “Excluir” só para `ADMIN` e confirme que o back também bloqueia um `USER`.

3. Faça o token expirar em 1 minuto e verifique que o front volta para o login sozinho.

4. Troque o `localStorage` por estado em memória e descreva o que mudou para o usuário.

Para ir além

- Projetos `SpringBoot/SecureLoginPUC_Matrix` e `React/SecureLoginPUC_React`
- react.dev/reference/react/createContext

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 08?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web